

Internship Report

Digital Electronics and Embedded Systems

Rugved Kshirsagar

Indian Institute of Science Education and Research-BHOPAL

July 9, 2026

Contents

1	Introduction	5
2	Objectives	6
3	Experiments	7
3.1	Experiment 1: Seven Segment Display Hardware Setup	7
3.1.1	Aim	7
3.1.2	Theory	7
3.1.3	Components Required	8
3.1.4	Procedure	8
3.1.5	Observation	8
3.1.6	Result	8
3.2	Experiment 2: Manual Control of Seven Segment Display	8
3.2.1	Aim	8
3.2.2	Theory	9
3.2.3	Procedure	9
3.2.4	Truth Table	9
3.2.5	Observation	9
3.2.6	Result	10
3.3	Experiment 3: Seven Segment Display using Arduino	10
3.3.1	Aim	10
3.3.2	Theory	10
3.3.3	Components Required	10
3.3.4	Pin Connections	11
3.3.5	Procedure	11
3.3.6	Arduino Program	11
3.3.7	Observation	12
3.3.8	Result	12
3.4	Experiment 4: BCD to Seven Segment Decoder using IC 7447	12
3.4.1	Aim	12
3.4.2	Theory	12

3.4.3	Components Required	13
3.4.4	Pin Connections	13
3.4.5	BCD Truth Table	14
3.4.6	Procedure	14
3.4.7	Observation	14
3.4.8	Result	14
3.5	Experiment 5: Software Implementation of the 7447 Decoder	15
3.5.1	Aim	15
3.5.2	Theory	15
3.5.3	Components Required	15
3.5.4	Working Principle	15
3.5.5	Procedure	16
3.5.6	Observation	16
3.5.7	Result	16
3.6	Experiment 6: Karnaugh Map Based Incrementing Decoder	16
3.6.1	Aim	16
3.6.2	Theory	17
3.6.3	Components Required	17
3.6.4	Working Principle	17
3.6.5	Procedure	17
3.6.6	Truth Table	18
3.6.7	Boolean Simplification	18
3.6.8	Observation	18
3.6.9	Applications	18
3.6.10	Result	19
4	Project 2: Finite State Machine using D Flip-Flops	20
4.1	Aim	20
4.2	Theory	20
4.3	Components Required	21
4.4	Design Procedure	21
4.5	State Transition Table	21
4.6	State Diagram	22
4.7	Working Principle	22
4.8	Observation	22
4.9	Result	22
5	Project 3: Digital Clock using ATmega328P	23
5.1	Aim	23

5.2	Theory	23
5.3	Components Required	24
5.4	Working Principle	24
5.4.1	Source Code	24
5.5	Applications	25
5.6	Observation	26
5.7	Result	26
6	Project 4: Scientific Calculator using Arduino	27
6.1	Aim	27
6.2	Introduction	27
6.3	Components Required	28
6.4	Working Principle	28
6.5	Matrix Keypad	28
6.6	LCD Interfacing	29
6.7	Functions Implemented	29
6.8	Software Design	29
6.9	Advantages	30
6.10	Applications	30
6.11	Observation	30
6.12	Result	30
7	Skills Acquired	32
8	Conclusion	33

Abstract

The internship was conducted to provide practical exposure to digital electronics, embedded systems and hardware implementation of digital logic. The training began with the setup of a Linux-based development environment and gradually progressed towards the implementation of digital circuits using discrete logic components and microcontrollers.

Initially, experiments involving seven-segment displays were performed to understand their construction, manual control, software control using Arduino and BCD decoding using the 7447 integrated circuit. Boolean logic simplification using Karnaugh Maps was also studied and implemented.

Subsequently, sequential circuits were explored through the implementation of decade counters using 7474 D flip-flops and finite state machines. Finally, complete embedded applications such as a digital clock using an ATmega328P microcontroller and a scientific calculator using Arduino, LCD display and a 5×5 matrix keypad were successfully developed.

The internship provided valuable practical experience in embedded programming, digital circuit design, debugging techniques and hardware-software integration.

Chapter 1

Introduction

Digital electronics forms the foundation of modern computing systems, embedded devices and communication equipment. While classroom learning provides the theoretical background required for understanding digital circuits, practical implementation plays an equally important role in understanding the behaviour of hardware.

The objective of this internship was to bridge the gap between theoretical knowledge and practical implementation through a sequence of progressively challenging experiments. Beginning with simple combinational logic circuits, the internship gradually introduced sequential logic, finite state machines and embedded systems using Arduino and ATmega microcontrollers.

Each experiment was carefully designed to strengthen the understanding of digital design principles while simultaneously improving debugging skills, circuit implementation techniques and programming ability.

The internship concluded with two complete embedded projects: a digital clock and a scientific calculator, which integrated concepts learned throughout the internship.

Chapter 2

Objectives

The primary objectives of the internship were:

- To understand the working principle of digital logic circuits.
- To gain practical experience with seven-segment displays.
- To interface Arduino boards with external hardware.
- To understand the operation of BCD-to-seven-segment decoder ICs.
- To simplify Boolean expressions using Karnaugh Maps.
- To design sequential circuits using D Flip-Flops.
- To implement finite state machines.
- To interface LCD displays and keypad matrices.
- To develop embedded applications using Arduino and ATmega328P.
- To improve debugging and hardware integration skills.

Chapter 3

Experiments

3.1 Experiment 1: Seven Segment Display Hardware Setup

3.1.1 Aim

To study the construction, pin configuration and hardware interfacing of a seven-segment display.

3.1.2 Theory

A seven-segment display is an electronic display device consisting of seven individual Light Emitting Diodes (LEDs) arranged in the shape of the number "8". By illuminating different combinations of these LEDs, decimal digits from 0 to 9 can be displayed.

The seven segments are labelled as a , b , c , d , e , f and g . An additional decimal point LED may also be provided.

The display used during the internship was interfaced using a breadboard and Arduino Uno. Power and ground connections were first verified before controlling the individual segments.

3.1.3 Components Required

Component	Quantity
Arduino Uno	1
Seven Segment Display	1
220 Ω Resistor	1
Breadboard	1
Jumper Wires	As Required

Table 3.1: Components required for Experiment 1

3.1.4 Procedure

1. The Arduino was connected to the breadboard.
2. The power rails of the breadboard were connected to the 5V and GND pins of the Arduino.
3. The seven-segment display was placed on the breadboard.
4. The common pin of the display was connected through a current limiting resistor.
5. All wiring connections were verified before powering the circuit.

3.1.5 Observation

The display was successfully powered and the pin configuration of the seven-segment display was verified.

3.1.6 Result

The hardware interfacing of the seven-segment display was completed successfully, providing the foundation for the subsequent experiments involving manual and software control.

3.2 Experiment 2: Manual Control of Seven Segment Display

3.2.1 Aim

To manually generate decimal digits by controlling the individual segments of a seven-segment display.

3.2.2 Theory

Each decimal digit is produced by illuminating a specific combination of the seven LED segments. In this experiment, the required segments were manually connected to logic LOW while the remaining segments were kept at logic HIGH. This helped in understanding how numerical digits are formed without using any decoder or software.

The experiment also introduced the concept of truth tables for display devices, where each combination of segment states corresponds to a unique decimal digit.

3.2.3 Procedure

1. The seven-segment display was connected to the breadboard.
2. Individual segment pins were connected to GND or left disconnected according to the desired digit.
3. The displayed digit was observed.
4. The procedure was repeated for all digits from 0 to 9.

3.2.4 Truth Table

Digit	a	b	c	d	e	f	g
0	0	0	0	0	0	0	1
1	1	0	0	1	1	1	1
2	0	0	1	0	0	1	0
3	0	0	0	0	1	1	0
4	1	0	0	1	1	0	0
5	0	1	0	0	1	0	0
6	0	1	0	0	0	0	0
7	0	0	0	1	1	1	1
8	0	0	0	0	0	0	0
9	0	0	0	0	1	0	0

Table 3.2: Segment combinations for decimal digits (active LOW)

3.2.5 Observation

The decimal digits were successfully generated by manually selecting the required segment combinations. The experiment demonstrated the relationship between binary control signals and the visual output of the display.

3.2.6 Result

The operation of the seven-segment display was verified successfully. Manual control of the display provided a clear understanding of segment activation and decimal digit generation.

3.3 Experiment 3: Seven Segment Display using Arduino

3.3.1 Aim

To interface a seven-segment display with an Arduino Uno and display decimal digits through software control.

3.3.2 Theory

Unlike manual hardware control, software control allows the state of each segment to be controlled programmatically using the Arduino's digital output pins. The Arduino sends HIGH or LOW signals to the corresponding segment pins according to the desired digit.

The program defines a function that accepts seven binary inputs corresponding to the seven segments of the display. Different combinations of these inputs generate the decimal digits from 0 to 9.

This method greatly simplifies the control of the display and forms the basis for implementing more complex embedded systems.

3.3.3 Components Required

Component	Quantity
Arduino Uno	1
Seven Segment Display	1
Breadboard	1
220 Ω Resistor	1
Jumper Wires	As Required
USB Cable	1

Table 3.3: Components used in Experiment 3

3.3.4 Pin Connections

Arduino Pin	Display Segment
D2	a
D3	b
D4	c
D5	d
D6	e
D7	f
D8	g

Table 3.4: Arduino to Seven Segment connections

3.3.5 Procedure

1. Connected the seven-segment display to the Arduino according to the specified pin configuration.
2. Configured the digital pins as output pins.
3. Wrote a function to control each display segment.
4. Uploaded the program to the Arduino Uno.
5. Verified the displayed output for different input combinations.

3.3.6 Arduino Program

```
void sevenseg(int a,int b,int c,
              int d,int e,int f,int g)
{
  digitalWrite(2,a);
  digitalWrite(3,b);
  digitalWrite(4,c);
  digitalWrite(5,d);
  digitalWrite(6,e);
  digitalWrite(7,f);
  digitalWrite(8,g);
}

void setup()
```

```

{
  pinMode(2,OUTPUT);
  pinMode(3,OUTPUT);
  pinMode(4,OUTPUT);
  pinMode(5,OUTPUT);
  pinMode(6,OUTPUT);
  pinMode(7,OUTPUT);
  pinMode(8,OUTPUT);
}

void loop()
{
  sevenseg(1,0,0,1,1,1,1);
}

```

3.3.7 Observation

The Arduino successfully generated the required logic levels for the seven-segment display. The displayed output matched the programmed digit.

3.3.8 Result

The seven-segment display was successfully controlled using Arduino. The experiment demonstrated the integration of digital hardware with embedded software and verified the correctness of the implemented program.

3.4 Experiment 4: BCD to Seven Segment Decoder using IC 7447

3.4.1 Aim

To study the operation of the 7447 BCD-to-Seven Segment Decoder IC and verify its functionality through hardware implementation.

3.4.2 Theory

The 7447 is a combinational logic integrated circuit that converts a 4-bit Binary Coded Decimal (BCD) input into the corresponding seven-segment display outputs. It eliminates

the need to manually generate the seven control signals required for displaying decimal digits.

The IC accepts four binary inputs labelled A, B, C and D, representing the BCD equivalent of decimal numbers from 0 to 9. Internally, the decoder generates the appropriate output pattern required to illuminate the corresponding segments of the display.

The outputs of the 7447 are active LOW and are intended to drive common-anode seven-segment displays.

3.4.3 Components Required

Component	Quantity
Arduino Uno	1
7447 Decoder IC	1
Seven Segment Display	1
Breadboard	1
Jumper Wires	As Required
220 Ω Resistors	7

Table 3.5: Components used in Experiment 4

3.4.4 Pin Connections

The outputs of the 7447 decoder were connected directly to the corresponding segments of the seven-segment display. The BCD inputs were provided manually through switches or jumper wires.

7447 Input	Description
A	Least Significant Bit
B	Second Bit
C	Third Bit
D	Most Significant Bit

Table 3.6: BCD Inputs of the 7447 Decoder

3.4.5 BCD Truth Table

Decimal	D	C	B	A
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

Table 3.7: BCD representation of decimal digits

3.4.6 Procedure

1. Inserted the 7447 IC on the breadboard.
2. Connected the seven-segment display to the decoder outputs.
3. Connected the supply voltage and ground.
4. Applied different BCD inputs through jumper wires.
5. Observed the corresponding decimal digit displayed on the seven-segment display.
6. Repeated the procedure for all decimal numbers from 0 to 9.

3.4.7 Observation

For every valid BCD input, the decoder generated the correct output pattern and the corresponding decimal digit was displayed correctly on the seven-segment display.

The experiment demonstrated the practical implementation of combinational logic using a dedicated decoder IC.

3.4.8 Result

The operation of the 7447 BCD-to-Seven Segment Decoder was successfully verified. The decoder correctly translated every BCD input into its corresponding decimal digit, demonstrating the practical implementation of combinational logic using integrated circuits.

3.5 Experiment 5: Software Implementation of the 7447 Decoder

3.5.1 Aim

To implement the functionality of the 7447 BCD-to-Seven Segment Decoder using Arduino software and verify the generated outputs.

3.5.2 Theory

The 7447 decoder performs a combinational logic operation that converts a 4-bit Binary Coded Decimal (BCD) input into the corresponding seven output signals required to drive a seven-segment display.

Instead of using the hardware decoder IC, the same functionality can be implemented through software. The Arduino reads the BCD input, evaluates the required output pattern, and generates the corresponding logic levels at its digital output pins.

This experiment demonstrates that combinational logic can be implemented either through dedicated hardware or by software running on a microcontroller.

3.5.3 Components Required

Component	Quantity
Arduino Uno	1
Seven Segment Display	1
Breadboard	1
220 Ω Resistors	7
Jumper Wires	As Required
USB Cable	1

Table 3.8: Components used in Experiment 5

3.5.4 Working Principle

The Arduino receives a 4-bit BCD input corresponding to the decimal digits 0 to 9. Based on the input, the program determines which of the seven display segments should be activated.

Unlike the previous experiment, no external decoder IC is used. All decoding is performed through software, thereby emulating the functionality of the 7447 integrated circuit.

3.5.5 Procedure

1. Connected the seven-segment display directly to the Arduino.
2. Configured the digital pins as outputs.
3. Wrote the decoding logic for the decimal digits.
4. Uploaded the program to the Arduino Uno.
5. Applied different BCD inputs.
6. Observed the displayed output.
7. Compared the output with that of the hardware 7447 decoder.

3.5.6 Observation

The software implementation successfully generated the same output patterns as the hardware 7447 decoder. Each BCD input correctly displayed the corresponding decimal digit on the seven-segment display.

No significant difference was observed between the outputs of the hardware and software implementations.

3.5.7 Result

The functionality of the 7447 BCD-to-Seven Segment Decoder was successfully implemented using Arduino software. The experiment demonstrated that combinational logic circuits can be realized either using dedicated hardware ICs or through embedded software with identical functionality.

3.6 Experiment 6: Karnaugh Map Based Incrementing Decoder

3.6.1 Aim

To design and implement an incrementing decoder using Karnaugh Map (K-map) simplification and verify its operation using Arduino.

3.6.2 Theory

Digital logic circuits can often be represented using Boolean expressions derived from their truth tables. However, these expressions may contain redundant terms that increase hardware complexity.

A Karnaugh Map (K-map) is a graphical technique used to minimize Boolean expressions. By grouping adjacent cells containing logic 1's, simplified expressions can be obtained with fewer logic gates.

In this experiment, an incrementing decoder was designed. The decoder accepts a Binary Coded Decimal (BCD) input representing the decimal digits 0 to 9 and generates the BCD representation of the next decimal number. Thus, an input of 3 produces an output of 4, while an input of 9 resets the output to 0.

The simplified Boolean expressions obtained using Karnaugh Maps were implemented on an Arduino and verified experimentally.

3.6.3 Components Required

Component	Quantity
Arduino Uno	1
Breadboard	1
Seven Segment Display	1
7447 Decoder IC	1
Jumper Wires	As Required
220 Ω Resistors	7

Table 3.9: Components used in Experiment 6

3.6.4 Working Principle

The input consists of a four-bit BCD number represented by the variables W , X , Y and Z .

For every valid input between 0 and 9, the circuit generates the BCD representation of the next decimal number.

The truth table was first prepared. Separate Karnaugh Maps were then drawn for each output bit. After minimization, the resulting Boolean expressions were implemented using Arduino.

3.6.5 Procedure

1. Prepared the truth table for the incrementing decoder.
2. Drew Karnaugh Maps for all four output bits.

3. Simplified the Boolean expressions.
4. Implemented the logic using Arduino.
5. Connected the Arduino outputs to the 7447 decoder.
6. Verified the displayed outputs for all decimal inputs.

3.6.6 Truth Table

W	X	Y	Z	D	C	B	A
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	0
0	0	1	0	0	0	1	1
0	0	1	1	0	1	0	0
0	1	0	0	0	1	0	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	1	1
0	1	1	1	1	0	0	0
1	0	0	0	1	0	0	1
1	0	0	1	0	0	0	0

Table 3.10: Truth table of the incrementing decoder

3.6.7 Boolean Simplification

Each output bit was minimized independently using Karnaugh Maps. The simplified Boolean expressions reduced the hardware complexity while preserving the desired functionality.

The minimized expressions were then translated into Arduino code using logical operators.

3.6.8 Observation

The Arduino successfully generated the incremented BCD outputs for every valid input. The outputs matched the theoretical truth table obtained from Karnaugh Map simplification.

The experiment demonstrated the effectiveness of Boolean minimization in reducing circuit complexity.

3.6.9 Applications

- Digital Counters

- Sequence Generators
- Finite State Machines
- Embedded Digital Systems
- Digital Display Controllers

3.6.10 Result

The incrementing decoder was successfully designed using Karnaugh Map simplification and implemented using Arduino. The experimental outputs matched the theoretical predictions for all valid BCD inputs, validating the correctness of the simplified Boolean expressions.

Chapter 4

Project 2: Finite State Machine using D Flip-Flops

4.1 Aim

To design and implement a Finite State Machine (FSM) using D Flip-Flops and verify its operation through hardware implementation.

4.2 Theory

A Finite State Machine (FSM) is a sequential logic circuit whose output depends on both the present inputs and the current state of the system. Unlike combinational logic circuits, an FSM stores information about previous inputs using memory elements such as flip-flops.

An FSM consists of three major components:

- State Register
- Next State Logic
- Output Logic

At every clock pulse, the state register stores the next state generated by the combinational logic. This process continues until the desired sequence of states is obtained.

Finite State Machines are widely used in digital systems because they provide an organized method of implementing sequential operations such as controllers, communication protocols, counters and processors.

4.3 Components Required

Component	Quantity
7474 D Flip-Flop IC	2
Breadboard	1
Push Button	1
LEDs	2
220 Ω Resistors	2
Jumper Wires	As Required
5 V Supply	1

Table 4.1: Components used for FSM implementation

4.4 Design Procedure

The implementation of the finite state machine involved the following steps:

1. Defining the required sequence of states.
2. Preparing the state transition table.
3. Determining the next-state equations.
4. Simplifying the Boolean expressions using Karnaugh Maps.
5. Implementing the simplified logic using D Flip-Flops.
6. Verifying the state transitions experimentally.

4.5 State Transition Table

The finite state machine changes its state only when a clock pulse is received.

Present State	Next State	Output
00	01	0
01	10	0
10	11	1
11	00	1

Table 4.2: Example State Transition Table

4.6 State Diagram

The corresponding state diagram consists of four states connected sequentially.

$$00 \rightarrow 01 \rightarrow 10 \rightarrow 11 \rightarrow 00$$

In the implemented circuit, every clock pulse causes the machine to transition to the next state according to the transition table.

4.7 Working Principle

Initially, the flip-flops are reset to the starting state.

The output of each flip-flop is continuously fed back into the combinational logic network.

The combinational logic calculates the next state based on the current state.

Whenever a positive clock edge is received, the D Flip-Flops store the calculated next state.

This process repeats continuously, producing the required sequence of states.

4.8 Observation

The designed FSM successfully changed states according to the predefined sequence. The LEDs connected to the outputs of the flip-flops indicated the current state after each clock pulse. The observed state transitions matched the theoretical state transition table.

4.9 Result

The finite state machine was successfully designed using D Flip-Flops and implemented on hardware. The experimental observations agreed with the theoretical state transitions, demonstrating the practical implementation of sequential logic circuits.

Chapter 5

Project 3: Digital Clock using ATmega328P

5.1 Aim

To design and implement a digital clock using the ATmega328P microcontroller, a crystal oscillator and four multiplexed seven-segment displays for displaying time in the HH:MM format.

5.2 Theory

A digital clock is an embedded system that continuously measures time and displays it in a human-readable format. In this project, the ATmega328P microcontroller acts as the central processing unit responsible for timekeeping, display control and synchronization.

The ATmega328P executes instructions using an external crystal oscillator which provides a stable clock signal. Accurate timing is essential for maintaining the correctness of the displayed time. Timer peripherals inside the microcontroller are configured to generate periodic interrupts, allowing seconds, minutes and hours to be updated precisely.

Since four seven-segment displays are used, driving each display independently would require a large number of I/O pins. To overcome this limitation, multiplexing is employed. In multiplexing, only one display is activated at a time while the segment data is supplied simultaneously. This process is repeated rapidly for all four displays, creating the illusion that all digits are continuously illuminated.

5.3 Components Required

Component	Quantity
ATmega328P Microcontroller	1
16 MHz Crystal Oscillator	1
22 pF Capacitors	2
Seven Segment Display	4
220 Ω Resistors	28
Breadboard	1
Power Supply (5 V)	1
Jumper Wires	As Required

Table 5.1: Components used in the Digital Clock

5.4 Working Principle

The ATmega328P initializes all hardware peripherals during startup. The crystal oscillator generates a stable clock signal, enabling accurate execution of instructions.

A hardware timer is configured to generate interrupts at fixed intervals. The interrupt service routine updates the seconds counter. Whenever the seconds reach 60, they are reset to zero and the minutes counter is incremented. Similarly, after 60 minutes the hour counter is incremented. After 24 hours the clock resets to 00:00.

The four seven-segment displays are refreshed sequentially using multiplexing. Each display is enabled for a very short duration before switching to the next display. Due to persistence of vision, the observer perceives all four digits as continuously illuminated.

5.4.1 Source Code

The following code initializes the I/O pins of the digital clock, configures the push buttons and repeatedly updates the clock while refreshing the display.

Listing 5.1: Initialization and main loop of the digital clock

```
const byte segPins[7] = {2, 3, 4, 5, 6, 7, 8};  
const byte digitPins[4] = {9, 10, 11, 12};  
  
const byte hourBtn = A0;  
const byte minuteBtn = A1;  
const byte startBtn = A2;
```

```

void setup()
{
    for (int i = 0; i < 7; i++)
        pinMode(segPins[i], OUTPUT);

    for (int i = 0; i < 4; i++)
        pinMode(digitPins[i], OUTPUT);

    pinMode(hourBtn, INPUT_PULLUP);
    pinMode(minuteBtn, INPUT_PULLUP);
    pinMode(startBtn, INPUT_PULLUP);
}

void loop()
{
    readButtons();
    updateClock();
    refreshDisplay();
}

```

5.5 Applications

- Digital clocks
- Electronic timers
- Industrial control systems
- Embedded instrumentation
- Home automation systems

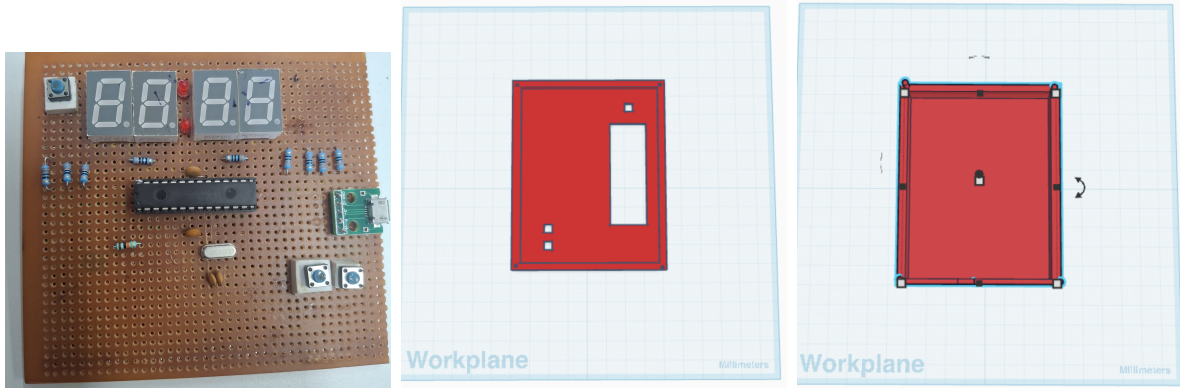


Figure 5.1: Different stages of the digital clock implementation.

5.6 Observation

The digital clock successfully displayed the current time using four seven-segment displays. The timer interrupt maintained accurate timekeeping while multiplexing ensured that all four displays appeared continuously illuminated without noticeable flicker.

The clock correctly updated hours and minutes according to the programmed logic.

5.7 Result

A digital clock was successfully implemented using the ATmega328P microcontroller, a crystal oscillator and four multiplexed seven-segment displays. The project demonstrated the practical use of timer interrupts, multiplexing techniques and embedded programming for real-time applications.

Chapter 6

Project 4: Scientific Calculator using Arduino

6.1 Aim

To design and implement a scientific calculator using an Arduino Uno, a 16×2 LCD display and a 5×5 matrix keypad capable of performing both arithmetic and scientific computations.

6.2 Introduction

Scientific calculators are embedded systems capable of evaluating mathematical expressions involving arithmetic, trigonometric and logarithmic functions. Unlike simple calculators, they require efficient algorithms for expression parsing and numerical computation.

In this project, an Arduino Uno was used as the processing unit. User input was provided through a 5×5 matrix keypad while the results were displayed on a 16×2 LCD module.

The implementation combined hardware interfacing with embedded programming to realize a compact scientific calculator.

6.3 Components Required

Component	Quantity
Arduino Uno	1
16×2 LCD Display	1
Push Buttons	25
Breadboard	1
10 k Ω Potentiometer	1
Jumper Wires	As Required
USB Cable	1

Table 6.1: Components used for the Scientific Calculator

6.4 Working Principle

The calculator accepts user input through a 5×5 matrix keypad. The Arduino continuously scans the keypad by activating one row at a time and reading the corresponding columns. Whenever a key press is detected, the Arduino determines its row and column position and identifies the pressed key.

The entered mathematical expression is displayed on the LCD. Once the equals key is pressed, the expression is evaluated and the result is displayed.

The LCD communicates with the Arduino using 4-bit mode, reducing the number of required I/O pins while maintaining reliable communication.

6.5 Matrix Keypad

The keypad consists of five rows and five columns.

Instead of requiring twenty-five independent input pins, matrix scanning reduces the requirement to only ten microcontroller pins.

The Arduino repeatedly performs the following sequence:

1. Activate one row.
2. Read all five columns.
3. Detect any pressed key.
4. Move to the next row.
5. Repeat continuously.

This technique provides efficient hardware utilization while allowing rapid user input.

6.6 LCD Interfacing

A standard 16×2 LCD module was used for displaying user input and calculated results.

The LCD operates in 4-bit mode where data is transferred through four data lines along with the Register Select (RS) and Enable (EN) control signals.

The display provides immediate visual feedback during expression entry and computation.

6.7 Functions Implemented

The calculator supports the following operations:

- Addition
- Subtraction
- Multiplication
- Division
- Trigonometric functions
- Inverse trigonometric functions
- Logarithmic functions
- Exponential functions
- Power functions
- Square root
- Factorial

6.8 Software Design

The software consists of four major modules:

1. Keypad Scanning
2. LCD Driver
3. Expression Parser
4. Mathematical Function Evaluation

The program continuously scans the keypad, updates the LCD display and evaluates the entered mathematical expressions whenever requested by the user.

6.9 Advantages

- Compact hardware implementation.
- Low cost.
- Expandable software architecture.
- Efficient keypad scanning.
- User-friendly interface.
- Supports both arithmetic and scientific calculations.

6.10 Applications

- Educational laboratories
- Engineering calculations
- Embedded system demonstrations
- Mathematical computations
- Portable calculation devices

6.11 Observation

The scientific calculator successfully accepted user input through the keypad, displayed expressions on the LCD and correctly evaluated arithmetic and scientific functions. The matrix scanning algorithm enabled reliable key detection while the LCD provided clear visual feedback.

6.12 Result

A scientific calculator was successfully implemented using an Arduino Uno, a 16×2 LCD display and a 5×5 matrix switches. The project demonstrated the integration of hardware interfacing, embedded programming and mathematical computation within a single embedded system.

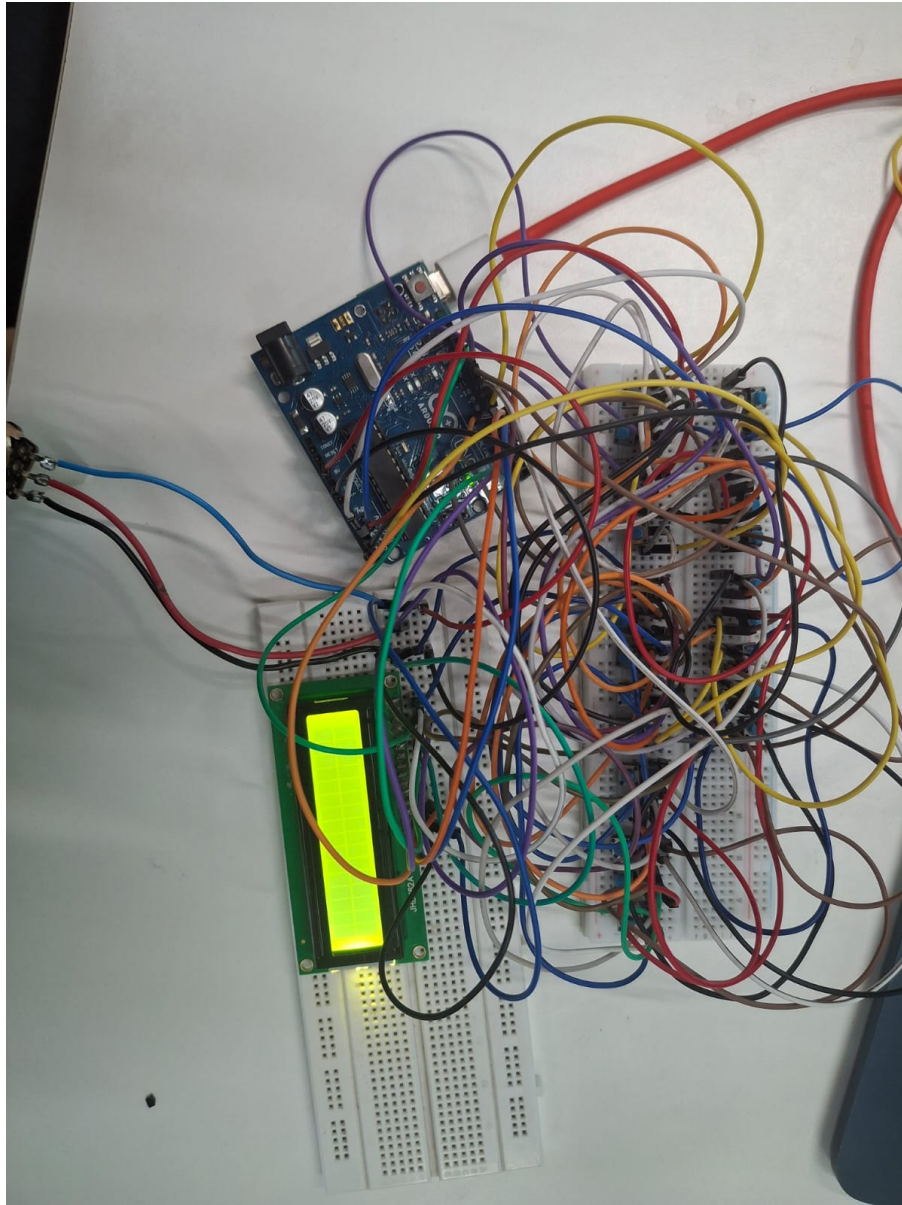


Figure 6.1: Scientific calculator implemented using an Arduino Uno, a 16×2 LCD display and a 5×5 matrix switches.

Chapter 7

Skills Acquired

The internship provided valuable practical exposure to digital electronics, embedded systems and hardware implementation. The following technical skills were developed during the course of the internship.

- Design and implementation of combinational logic circuits.
- Simplification of Boolean expressions using Karnaugh Maps.
- Design of sequential circuits using D Flip-Flops.
- Implementation of finite state machines.
- Interfacing of seven-segment displays.
- Usage of the 7447 BCD-to-Seven Segment Decoder.
- Arduino programming using Embedded C.
- Programming of the ATmega328P microcontroller.
- Interfacing LCD modules.
- Matrix keypad scanning techniques.
- Hardware debugging and circuit verification.
- Breadboard prototyping.
- Embedded system design methodology.
- Documentation using $\text{\LaTeX}$.

Chapter 8

Conclusion

The internship successfully bridged the gap between theoretical concepts in digital electronics and their practical implementation through a series of laboratory experiments and embedded system projects.

Beginning with the study of seven-segment displays and combinational logic circuits, the internship gradually introduced sequential logic, Karnaugh Map simplification, flip-flop based circuit design and finite state machines. These concepts were then integrated into larger embedded applications including a digital clock and a scientific calculator.

The internship significantly improved understanding of digital logic design, embedded programming, hardware interfacing and systematic debugging techniques. It also provided valuable experience in translating theoretical concepts into working hardware implementations.

Overall, the internship served as an excellent introduction to embedded systems and digital electronics while strengthening practical engineering skills required for future academic and industrial projects.